

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**). The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

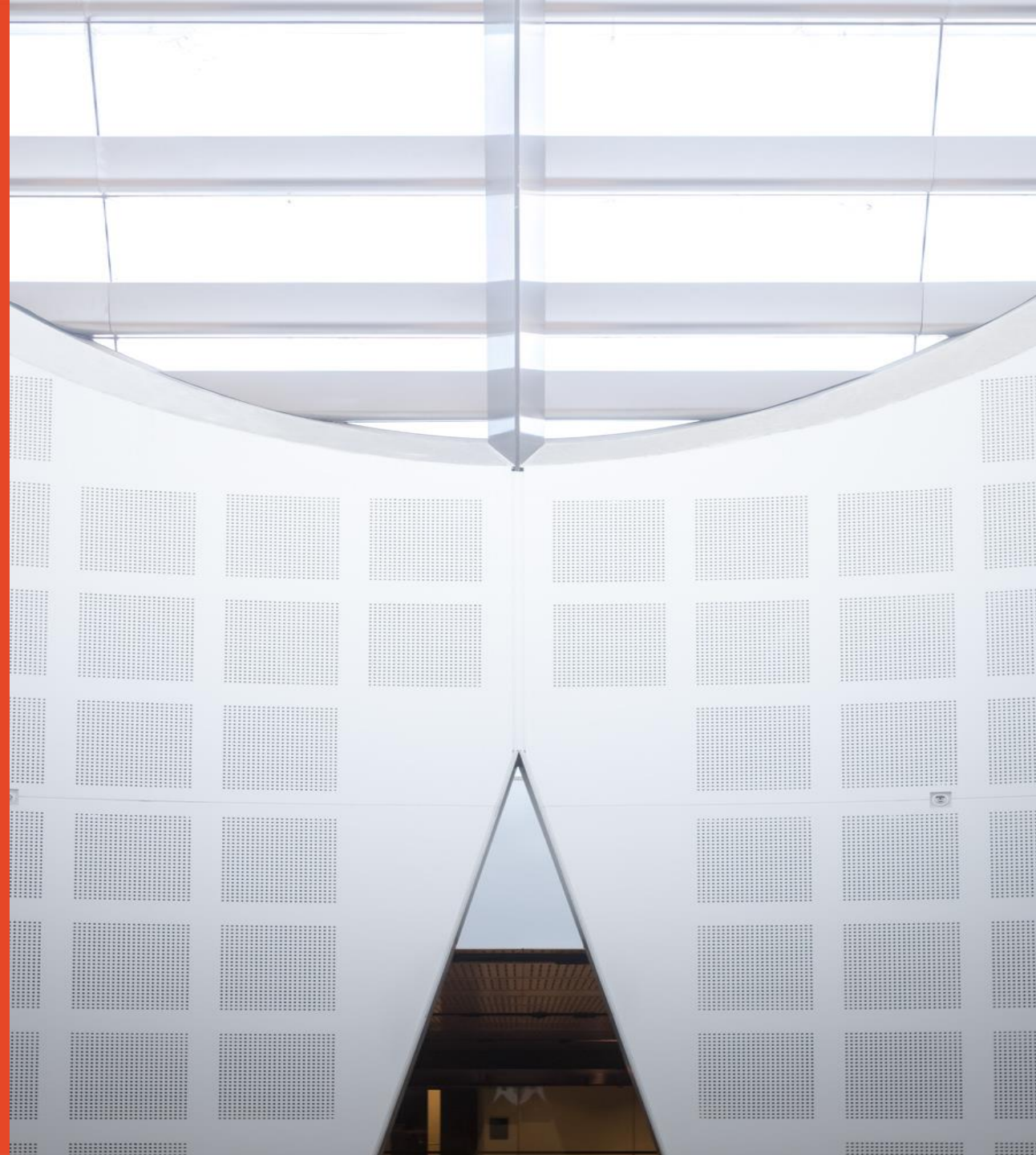
COMPx270: Randomised and Advanced Algorithms

Lecture 4: Derandomisation

Clément Canonne
School of Computer Science



THE UNIVERSITY OF
SYDNEY



A question

You have a **randomised** algorithm **A** which runs in time **$T(n)$** and solves task X (say, **decision** problem) with probability .99. Is there a **deterministic** algorithm **B** which solves X and runs in time...

- $O(T(n))$
- $\text{poly}(T(n))$
- $\exp(T(n))$
- No/we don't know

A question

You have a **randomised** algorithm **A** which runs in time **$T(n)$** and solves task X (say, **decision** problem) with probability .99. Is there a **deterministic** algorithm **B** which solves X and runs in time...

- $O(T(n))$?
- $\text{poly}(T(n))$?
- $\exp(T(n))$ ✓
- No/we don't know

An answer?

That's **complicated**. This is what **derandomization** asks, and there is a lot of work on this: **one of the major unsolved questions in theoretical computer science.**

P v. BPP

Let's not stop here though

We know how to **derandomize** **some** algorithms, and there are **some** general techniques.

Method 1: PRNG

The goal is to reduce the amount of randomness required, by generating a lot of "good enough" pseudorandom bits: good enough to **fool** the algorithm.

Fact. Under some "plausible assumptions," there exist PRNGs $G: \{0, 1\}^{\ell} \rightarrow \{0, 1\}^n$ · $A = \{\text{all poly-time algorithms}\}$ with $\ell = O(\log n)$.

Method 1: PRNG

Why is that useful?

- Random bits don't grow on trees!
- Derandomisation (method 2)

Why is this bad?

- Conditional (under assumptions)

Method 2: Brute force 💪

If the algorithm uses a small number of random seeds, **check 'em all**.

Method 2: Brute force 🦊

If the algorithm uses a small number of random seeds, **check 'em all**.

Require: Input x

1: **for all** $r \in \{0, 1\}^R$ **do**

2: $y \leftarrow A(x; r)$

▷ Run A on x with randomness r

3: **if** $V(x, y) = 1$ **then**

▷ Verify if y is a good solution

4: **return** y

▷ If so, we are done

Details.

Method 2: Brute force

What if verifying is **hard**?

Method 2: Brute force 🦾

What if verifying is **hard**?

- Majority vote!
- Median trick!

Method 2: Brute force 💪

What if the algorithm does **not** use a small number of random bits?

Method 2: Brute force 💪

What if the algorithm does **not** use a small number of random bits?

Well, these PRNGs can come in handy...

Method 2: Brute force 🦾

What if the algorithm does **not** use a small number of random bits?

Or (sometimes) we can reduce the randomness by carefully looking at the proof.

Method 2: Brute force 🦖 via pairwise independence

Derandomizing Max-Cut

MAX-CUT: Given an (undirected) graph $G = (V, E)$ on n vertices and m edges, output a cut (A, B) (partition of V) *maximising* the number $c(A, B)$ of edges between A and B .

(It's NP-Hard)

Method 2: Brute force 🦖 via pairwise independence

Derandomizing Max-Cut

MAX-CUT: Given an (undirected) graph $G = (V, E)$ on n vertices and m edges, output a cut (A, B) (partition of V) *maximising* the number $c(A, B)$ of edges between A and B .

But we can get a $\frac{1}{2}$ -approximation!

Method 2: Brute force 🐢 via pairwise independence

```
1:  $(A, B) \leftarrow (\emptyset, \emptyset)$ 
2: for all  $v \in V$  do
3:    $X_v \leftarrow \text{Bern}(1/2)$            ▷ Independent of previous choices
4:   if  $X_v = 1$  then add  $v$  to  $A$ 
5:   else add  $v$  to  $B$ 
6: return  $(A, B)$ 
```

Method 2: Brute force 🦖 via pairwise independence

Theorem.

$$\mathbb{E}[c(A, B)] \geq \frac{1}{2} m \geq \frac{1}{2} \text{OPT}(G) .$$

Proof.

Method 2: Brute force 🦖 via pairwise independence

Theorem. This can be derandomised.

Method 2: Brute force 🦖 via pairwise independence

Theorem. This can be derandomised.

```
1:  $(A, B) \leftarrow (\emptyset, \emptyset)$ 
2: for all  $v \in V$  do
3:    $X_v \leftarrow \text{Bern}(1/2)$            ▷ Independent of previous choices
4:   if  $X_v = 1$  then add  $v$  to  $A$ 
5:   else add  $v$  to  $B$ 
6: return  $(A, B)$ 
```

Method 2: Pairwise independent hash functions

Definition 22.1. A family of functions $\mathcal{H} \subseteq \{h: \mathcal{X} \rightarrow \mathcal{Y}\}$ is a *family of pairwise independent hash functions*, or a *strongly universal hash family*, if, for every $x, x' \in \mathcal{X}$ with $x \neq x'$ and every $y, y' \in \mathcal{Y}$,

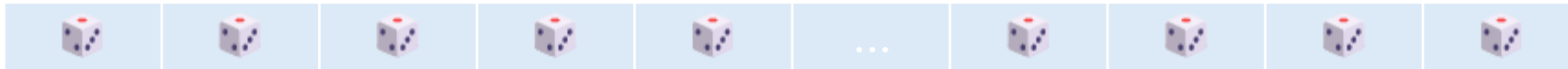
$$\Pr_{h \sim \mathcal{H}} [h(x) = y, h(x') = y'] = \frac{1}{|\mathcal{Y}|^2}$$

where the probability is over the uniformly random choice of $h \in \mathcal{H}$.

Method 2: Pairwise independent hash functions

For instance, $\mathcal{X} = V$ (vertices) = $\{1, 2, \dots, n\}$, and $\mathcal{Y} = \{0, 1\}$. The algorithm looks up a "random tape" filled with random bits it'll use.

Fully random: pick each value independently at random 



Hash function: pick $h: \{1, 2, \dots, n\} \rightarrow \{0, 1\}$ from $\mathcal{H}$ at random  **once**

$h(1)$	$h(2)$	$h(3)$	$h(4)$	$h(5)$	...	$h(n-3)$	$h(n-2)$	$h(n-1)$	$h(n)$
--------	--------	--------	--------	--------	-----	----------	----------	----------	--------

Method 2: Pairwise independent hash functions

Fact. Small families of pairwise independent hash functions exist.

Method 2: Pairwise independent hash functions

Fact. Small families of pairwise independent hash functions exist.

Proof of derandomization claim.

```
1:  $(A, B) \leftarrow (\emptyset, \emptyset)$ 
2: for all  $v \in V$  do
3:    $X_v \leftarrow \text{Bern}(1/2)$  ▷ Independent of previous choices
4:   if  $X_v = 1$  then add  $v$  to  $A$ 
5:   else add  $v$  to  $B$ 
6: return  $(A, B)$ 
```

(Important) Fact. If $\mathbb{E}[X]$ exists, then $\Pr[X \geq \mathbb{E}[X]] > 0$.

Proof.

Method 2: Brute force 🦖 via pairwise independence

Theorem. There exists a deterministic $\frac{1}{2}$ -approximation algorithm for Max-CUT which runs in time $O(n(m+n))$.

Method 3: The Method of Conditional Expectations

Idea: sequentially do the **greedy** choice. Sometimes it works!

Method 3: The Method of Conditional Expectations

Idea: sequentially do the **greedy** choice. Sometimes it works!

```
1:  $(A, B) \leftarrow (\emptyset, \emptyset)$ 
2: for all  $v \in V$  do
3:    $X_v \leftarrow \text{Bern}(1/2)$             $\triangleright$  Independent of previous choices
4:   if  $X_v = 1$  then add  $v$  to  $A$ 
5:   else add  $v$  to  $B$ 
6: return  $(A, B)$ 
```

Details.

Method 3: The Method of Conditional Expectations

Theorem. There exists a deterministic $\frac{1}{2}$ -approximation algorithm for Max-CUT which runs in time $O(mn)$.

Derandomisation: summary

- PRNG
- Brute-Force
- Pairwise (k -wise) independence
- Method of Conditional Expectations

(there is more!)

Bonus: The Probabilistic Method

"We can prove things exist without knowing how to build them."

(also can be derandomised, sometimes)